



Sum of Elements (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity
- Alternate Solution
 - Time complexity
 - Space complexity

Problem Statement

Given an array, find the sum of all numbers between the K1'th and K2'th smallest elements of that array.

Example 1:

Input: [1, 3, 12, 5, 15, 11], and K1=3, K2=6

Output: 23

Explanation: The 3rd smallest number is 5 and 6th smallest number 15. The sum of numbers coming between 5 and 15 is 23 (11+12).

Example 2:

Input: [3, 5, 8, 7], and K1=1, K2=4

Output: 12

Explanation: The sum of the numbers between the 1st smallest number (3) and the 4th smallest number (8) is 12 (5+7).

Try it yourself

Try solving this question here:



Java



Python3



JS



C++

```
1 import java.util.*;
2
3 class SumOfElements {
4
5     public static int findSumOfElements(int[] nums, int k1, int k2) {
6         // TODO: Write your code here
7     }
8 }
```



```

6 // TODO: write your code here
7 return -1;
8 }
9
10 public static void main(String[] args) {
11     int result = SumOfElements.findSumOfElements(new int[] { 1, 3, 12, 5, 15, 11 }, 3, 6);
12     System.out.println("Sum of all numbers between k1 and k2 smallest numbers: " + result);
13
14     result = SumOfElements.findSumOfElements(new int[] { 3, 5, 8, 7 }, 1, 4);
15     System.out.println("Sum of all numbers between k1 and k2 smallest numbers: " + result);
16 }
17 }
18

```

Run

Save

Reset



Solution

This problem follows the [Top 'K' Numbers](#) pattern, and shares similarities with [Kth Smallest Number](#).

We can find the sum of all numbers coming between the K1'th and K2'th smallest numbers in the following steps:

1. First, insert all numbers in a min-heap.
2. Remove the first **k1** smallest numbers from the min-heap.
3. Now take the next **k2-k1-1** numbers out of the heap and add them. This sum will be our required output.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1 import java.util.*;
2
3 class SumOfElements {
4
5     public static int findSumOfElements(int[] nums, int k1, int k2) {
6         PriorityQueue<Integer> minHeap = new PriorityQueue<Integer>((n1, n2) -> n1 - n2);
7         // insert all numbers to the min heap
8         for (int i = 0; i < nums.length; i++)
9             minHeap.add(nums[i]);
10
11         // remove k1 small numbers from the min heap
12         for (int i = 0; i < k1; i++)
13             minHeap.poll();
14
15         int elementSum = 0;
16         // sum next k2-k1-1 numbers
17         for (int i = 0; i < k2 - k1 - 1; i++)
18             elementSum += minHeap.poll();
19
20         return elementSum;
21     }
22
23     public static void main(String[] args) {
24         int result = SumOfElements.findSumOfElements(new int[] { 1, 3, 12, 5, 15, 11 }, 3, 6);
25         System.out.println("Sum of all numbers between k1 and k2 smallest numbers: " + result);
26
27         result = SumOfElements.findSumOfElements(new int[] { 3, 5, 8, 7 }, 1, 4);
28         System.out.println("Sum of all numbers between k1 and k2 smallest numbers: " + result);
29     }
30 }

```

31

Run

Save

Reset



Time complexity

Since we need to put all the numbers in a min-heap, the time complexity of the above algorithm will be $O(N * \log N)$ where 'N' is the total input numbers.

Space complexity

The space complexity will be $O(N)$, as we need to store all the 'N' numbers in the heap.

Alternate Solution

We can iterate the array and use a max-heap to keep track of the top K2 numbers. We can, then, add the top $K2 - K1 - 1$ numbers in the max-heap to find the sum of all numbers coming between the K1'th and K2'th smallest numbers. Here is what the algorithm will look like:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class SumOfElements {
4
5      public static int findSumOfElements(int[] nums, int k1, int k2) {
6          PriorityQueue<Integer> maxHeap = new PriorityQueue<Integer>((n1, n2) -> n2 - n1);
7          // keep smallest k2 numbers in the max heap
8          for (int i = 0; i < nums.length; i++) {
9              if (i < k2 - 1) {
10                 maxHeap.add(nums[i]);
11             } else if (nums[i] < maxHeap.peek()) {
12                 maxHeap.poll(); // as we are interested only in the smallest k2 numbers
13                 maxHeap.add(nums[i]);
14             }
15         }
16
17         // get the sum of numbers between k1 and k2 indices
18         // these numbers will be at the top of the max heap
19         int elementSum = 0;
20         for (int i = 0; i < k2 - k1 - 1; i++)
21             elementSum += maxHeap.poll();
22
23         return elementSum;
24     }
25
26     public static void main(String[] args) {
27         int result = SumOfElements.findSumOfElements(new int[] { 1, 3, 12, 5, 15, 11 }, 3, 6);
28         System.out.println("Sum of all numbers between k1 and k2 smallest numbers: " + result);
29
30         result = SumOfElements.findSumOfElements(new int[] { 3, 5, 8, 7 }, 1, 4);
31         System.out.println("Sum of all numbers between k1 and k2 smallest numbers: " + result);
32     }
33 }

```

Run

Save

Reset



Time complexity

Since we need to put only the top K^2 numbers in the max-heap at any time, the time complexity of the above algorithm will be $O(N * \log K^2)$.



Space complexity

The space complexity will be $O(K^2)$, as we need to store the smallest ' K^2 ' numbers in the heap.

[< Back](#)[✔ Mark As Completed](#)[Next >](#)[Maximum Distinct Elements \(medium\)](#)[Rearrange String \(hard\)](#)